COMP 418/518: Homework 3

[Weight of homework: 10% of the final grade]

Authors (fill in your names and NetIDs here)

Due on March 11, 2026
(released on February 27, 2026)

Instructions: For this homework, you should work in groups of 3–4 people (i.e., the groups that have been declared and accepted). Only one member of the group should submit on Gradescope. All group members must be included in the submission on the Gradescope system (otherwise no credit will be given). Also specify in your submission file all the members of your group (name and NetID) using the `\authors` macro. The submission must be typed in LaTeX using the provided template. The use of AI systems (including, but not limited to, ChatGPT, Gemini, Copilot, and Claude) is entirely allowed. No discussion or collaboration is allowed outside of the group. Late submissions are not accepted.

You should provide *pseudocode* for all the algorithms that you describe.

LaTeX Source: You must upload the LaTeX source for your submission on Gradescope. Your submission must contain everything that is needed to build the PDF file.

Grading: We will give a 5% grade bonus to students taking COMP 418. The homework will be graded based on correctness, completeness, quality of the solutions, and ***quality of writing***.

Guidelines:

- Do not use ellipses in your definitions or your proofs. Instead, use recursive definitions and induction.
- Provide a careful justification of every step in your proofs.
- If there are logical gaps, unsound mathematical reasoning, false/unclear statements, or any other mistakes, then points will be taken off accordingly.
- If a proof is unreadable, then it will receive 0 points. A proof should always be understandable and well written, because the goal is to provide a *convincing* mathematical argument.

Notation & remarks: For strings $x, y \in A^*$, we say that x is a *prefix* of y, and write $x \leq y$, if $xz = y$ for some $z \in A^*$. For example, ab is a prefix of $abcde$ because $ab \cdot cde = abcde$. Notice that if $x \leq y$ then there is a *unique* extension $z \in A^*$ such that $xz = y$. We write $x^{-1}y$ to denote this unique extension. For a string $x \in A^*$, we write $|x|$ to denote its length. For all strings $x, y \in A^*$, it holds that $|x \cdot y| = |x| + |y|$. String concatenation satisfies the following *left cancellation* property: for all $x, y, z \in A^*$, $xy = xz$ implies that $y = z$.

For sets X and Y, we write $X \to Y$ to denote the set of all functions from X to Y.

Problem 1 [5 points]

Let A be a set. Show that the prefix relation $\leq$ on A^* is a partial order (reflexive, antisymmetric and transitive).

Solution for 1

Fill me in.

Problem 2: Denotational Semantics [35 points]

In the lecture of February 20 we discussed denotations (functions) of type $A^+ \to B$ and of type $A^+ \to B^+$. Recall that a function $f : A^+ \to B$ describes a streaming computation in the following way: for an input history $x \in A^+$, the value $f(x)$ is the last output item after consuming x. We call f an *incremental* denotation, because it gives us the output increment. A function $\varphi : A^+ \to B^+$ can sometimes describe a streaming computation in the following way: for an input history $x \in A^+$, the value $\varphi(x)$ is the overall output history after consuming x. We call φ a *cumulative* denotation, because it gives us the cumulative (overall) output.

Remark: In this problem, we are only concerned with streaming computations that (i) produce exactly one output item per step (i.e., for each input item), and (ii) do *not* produce any output at the very beginning (i.e., before the first input item is seen).

Part 2A [5 points]

Define an injective function lift from the set $A^+ \to B$ to the set $A^+ \to B^+$. That is, $\mathsf{lift} : (A^+ \to B) \to (A^+ \to B^+)$ and lift is injective. The function lift should convert an incremental denotation to a cumulative denotation. Provide a proof that lift is injective.

Solution for 2A

Fill me in.

Part 2B [15 points]

For all sets A and B, specify a subset $\mathcal{S}(A, B) \subseteq (A^+ \to B^+)$ so that lift is a one-to-one correspondence (bijection) from $A^+ \to B$ to $\mathcal{S}(A, B)$. Provide a proof that lift is a bijection.

Solution for 2B

Fill me in.

Part 2C [15 points]

Recall the operation $\gg$ (serial composition) on incremental denotations from the lecture of February 20. Below is a description of how $\gg$ is applied to ensure that the types match up appropriately:

$$\frac{f : A^+ \to B \qquad g : B^+ \to C}{f \gg g : A^+ \to C}$$

The above notation is sometimes referred to as a *type rule* in the context of programming languages.

(1) Define a corresponding operation $\ggg$ (serial composition)[1] on cumulative denotations with the following type rule:

$$\frac{\varphi : A^+ \to B^+ \qquad \psi : B^+ \to C^+}{\varphi \ggg \psi : A^+ \to C^+}$$

(2) Show that $\varphi \ggg \psi \in \mathcal{S}(A, C)$ whenever $\varphi \in \mathcal{S}(A, B)$ and $\psi \in \mathcal{S}(B, C)$.

(3) Moreover, show that $\mathsf{lift}(f \gg g) = \mathsf{lift}(f) \ggg \mathsf{lift}(g)$ for all $f : A^+ \to B$ and $g : B^+ \to C$.

[1] We use the notation $\ggg$ to differentiate more easily with serial composition on incremental denotations.

Solution for 2C

Fill me in.

Problem 3: Pipeline Fusion [60 points]

Assume that we have the same (restricted) programming model for streaming computations that we considered in Problem 2.

Part 3A [4 points]

Consider the incremental denotation $f : \mathbb{Z}^+ \to \mathbb{Z}$, defined by $f(x) = x$ and $f(uxy) = y - x$ for every $u \in \mathbb{Z}^*$ and $x, y \in \mathbb{Z}$. Give a description in English of the computation that f describes and provide an implementation using pseudocode. Is your implementation stateful or stateless?

Solution for 3A

The denotation f computes the discrete first difference of the input stream, with a special case at the first step:

$$\forall x \in \mathbb{Z},\ f(x) = x \quad \text{and} \quad \forall u \in \mathbb{Z}^*,\ \forall x, y \in \mathbb{Z},\ f(uxy) = y - x.$$

So the output is the current value minus the previous value (except at the first item, where the output is the item itself). The implementation is *stateful*, because it must remember the previous input item.

```
// Implementation of f
// State:
hasPrev ∈ {false, true}, initially false
prev ∈ ℤ (value irrelevant while hasPrev = false)
Function Update(ℤ x):
    if hasPrev = false then
        // first input item
        hasPrev ← true
        prev ← x
        return x
    ℤ out ← x − prev
    prev ← x
    return out
```

Part 3B [3 points]

Consider the incremental denotation $g : \mathbb{Z}^+ \to \mathbb{Z}$, defined by $g(x) = x$ and $g(ux) = g(u) + x$ for every $u \in \mathbb{Z}^+$ and $x \in \mathbb{Z}$. Give a description in English of the computation that g describes and provide an implementation using pseudocode. Is your implementation stateful or stateless?

Solution for 3B

The denotation g computes the running prefix sum of the input stream:

$$\forall x \in \mathbb{Z},\ g(x) = x \quad \text{and} \quad \forall u \in \mathbb{Z}^+,\ \forall x \in \mathbb{Z},\ g(ux) = g(u) + x.$$

So at each step, the output is the total sum of all items seen so far. The implementation is *stateful*, because it must remember the running sum.

```
// Implementation of g
// State:
ℤ sum, initially 0
Function Update(ℤ y):
    sum ← sum + y
    return sum
```

Part 3C [8 points]

Let `F` be your implementation of f (from Part 3A) and `G` be your implementation of g (from Part 3B). Provide a single streaming algorithm that simulates the execution of the following pipeline:

$$\xrightarrow{\mathbb{Z}} \boxed{\texttt{F}} \xrightarrow{\mathbb{Z}} \boxed{\texttt{G}} \xrightarrow{\mathbb{Z}}$$

Use the "fusion" construction what we discussed on February 18. How many arithmetic operations does your implementation perform per step (i.e., for every input data item)?

Solution for 3C

Using fusion, we combine the state of `F` and `G` into one machine state. The fused state is the product of the two states:

$$(hasPrev, prev) \text{ for } \texttt{F} \quad \text{and} \quad sum \text{ for } \texttt{G}.$$

On each step we first compute the output of `F`, then use it to update `G`.

```
// Pipeline(F, G)
// State:
hasPrev ∈ {false, true}, initially false
prev ∈ ℤ (value irrelevant while hasPrev = false)
ℤ sum, initially 0
Function Update(ℤ x):
    if hasPrev = false then
        hasPrev ← true
        prev ← x
        ℤ d ← x
    else
        ℤ d ← x − prev
        prev ← x
    sum ← sum + d
    return sum
```

This fused implementation performs one subtraction and one addition on every step after the first step. At the first step, it performs one addition. Therefore the worst-case number of arithmetic operations per step is 2.

Part 3D [15 points]

The implementation "`Pipeline(F, G)`" that you provided in Part 3C is one out of many streaming algorithms that compute the incremental denotation $f \gg g : \mathbb{Z}^+ \to \mathbb{Z}$. Is it possible to provide a more efficient implementation of $f \gg g$ (i.e., one that uses less memory and performs fewer operations per step)? If your answer is "yes", provide the most efficient implementation you can think of and justify its correctness.

Solution for 3D

Yes. A strictly more efficient implementation exists.

Let the input history have length n, and let x_i denote the i-th input item. Define d_i as the i-th output of `F`:

$$d_1 = x_1 \quad \text{and} \quad d_i = x_i - x_{i-1} \text{ for all integers } i \text{ with } 2 \le i \le n.$$

Then the output of `G` at step n is

$$\sum_{i=1}^{n} d_i = x_1 + \sum_{i=2}^{n} (x_i - x_{i-1}) = x_n.$$

So for every history of length n, the value of $f \gg g$ is the last input item x_n. This is exactly the identity incremental denotation on $\mathbb{Z}^+$.

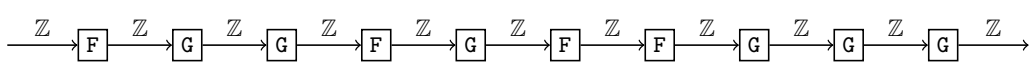
Hence an optimal implementation is:

$$\texttt{Update}(x)\ \texttt{return}\ x.$$

This implementation is stateless, uses less memory than `Pipeline(F,G)`, and performs zero arithmetic operations per step.

Part 3E [30 points]

Let `F` be your implementation of f (from Part 3A) and `G` be your implementation of g (from Part 3B). Suppose that P is a pipeline in which every stage is either `F` or `G` and there are $k \ge 0$ more `G` stages than `F` stages. For example, the pipeline



has 6 `G` stages and 4 `F` stages, which means that $k = 2$. Provide an efficient implementation of the same computation that P performs. You can use $k \ge 0$ as a parameter in your implementation. Justify the correctness of your implementation.

Solution for 3E

Let $\mathbf{id} : \mathbb{Z}^+ \to \mathbb{Z}$ be the identity incremental denotation, $\mathbf{id}(ux) = x$.

From Part 3D and the symmetric calculation, we have both cancellation laws:

$$f \gg g = \mathbf{id} \quad \text{and} \quad g \gg f = \mathbf{id}.$$

Consider any pipeline word over $\{\texttt{F}, \texttt{G}\}$. Repeatedly delete adjacent pairs `FG` or `GF`. Each deletion preserves denotation (by the equalities above) and shortens the word by 2, so the process terminates. The remaining reduced word has no adjacent opposite symbols. Since the alphabet has only two

symbols, the reduced word must be either $\mathtt{F}^m$ or $\mathtt{G}^m$ for some $m \geq 0$. Because the original pipeline has $k \geq 0$ more `G` stages than `F` stages, the reduced form is $\mathtt{G}^k$. Therefore, P is equivalent to k serial copies of `G`.

So we only need an implementation for $\mathtt{G}^k$.

```
// Pipeline P with k ≥ 0
// State:
Array S indexed by integers i
  with 1 ≤ i ≤ k
for i ← 1 to k do
    S[i] ← 0
Function Update(ℤ x):
    if k = 0 then
        return x
    S[1] ← S[1] + x
    for i ← 2 to k do
        S[i] ← S[i] + S[i − 1]
    return S[k]
```

Correctness of the implementation. For each integer i with $1 \leq i \leq k$, let $G_i(n)$ be the output of the i-th `G` stage at step n in a pipeline of k `G` stages. Also set $G_0(n) = x_n$. The stage recurrence is

$$G_i(n) = G_i(n-1) + G_{i-1}(n), \quad G_i(0) = 0.$$

The algorithm maintains the invariant $S[i] = G_i(n)$ after processing step n.

Base case ($n = 1$): after one update, $S[1] = x_1 = G_1(1)$, and then each loop update gives $S[i] = S[i] + S[i-1] = 0 + G_{i-1}(1) = G_i(1)$.

Induction step: assume after step $n-1$, $S[i] = G_i(n-1)$ for all i. At step n, the update for $S[1]$ yields

$$S[1] \leftarrow S[1] + x_n = G_1(n-1) + G_0(n) = G_1(n).$$

Then for each $i \geq 2$, after $S[i-1]$ has been updated to $G_{i-1}(n)$,

$$S[i] \leftarrow S[i] + S[i-1] = G_i(n-1) + G_{i-1}(n) = G_i(n).$$

Thus the invariant holds for step n. Therefore, after every step, the returned value $S[k]$ equals the output of $\mathtt{G}^k$, which equals the output of pipeline P.

The implementation uses k integer state variables and performs exactly k additions per step when $k > 0$ (and zero additions when $k = 0$).